CSA3023/CSE3023/CSM3023
WEB-BASED APPLICATION DEVELOPMENT

Lab Modules

Wan Nural Jawahir Hj Wan Yussof, Rabiei Mamat,
Mohd Arizal Shamsil Mat Rifin
& Muhammad Syaffiq Hazari

Contents

List of Figures

Lab Module 2

SERVLET: DATA SHARING AND DATABASE MANAGEMENT

2.1 Objectives

The primary goal of this lab is to familiarize students with dynamic web application development using Java Servlets and JDBC. By the end of this session, students should be able to:

1. **Implement Data Sharing in Servlet**: Share data across different components of a web application using `request`, `session`, or `ServletContext` attributes (e.g., `setAttribute()` and `getAttribute()`).

2. **Creating a table in MySQL Database**: Execute SQL `CREATE TABLE` statements to define the database schema, data types, and constraints for storing application data.

3. **Setting the Environment of Web Application for Database Connection**: Configure the JDBC driver, define the database URL, and set up the necessary credentials to establish a successful connection between the Java application and MySQL.

4. **Using Servlets for Database CRUD Operations**: Implement Create, Read, Update, and Delete operations within Servlet methods (such as `doGet` and `doPost`) by executing SQL queries via the JDBC API (`PreparedStatement` and `ResultSet`).

2.2 Introduction

Java Servlets are robust server-side components used to create dynamic, platform-independent web applications. When combined with Java Database Connectivity (JDBC), Servlets empower developers to interact seamlessly with relational databases, allowing the application to process complex business logic, manage data persistence, and generate dynamic responses based on client requests.

This lab focuses on the core mechanics of integrating a Java web application with a MySQL database. Students will explore the essential steps to configure the database connection environment, create database tables, and implement effective data sharing strategies across Servlet components. Furthermore, the lab emphasizes practical application by guiding students through the development of fully functional Create, Read, Update, and Delete (CRUD) operations using the JDBC API.

2.3 Software Requirements

To successfully complete the tasks outlined in this manual, the following software environment is required:

Software	Description
Java Development Kit (JDK)	Version 8 or higher (required to compile Java classes and Servlets).
Integrated Development Environment (IDE)	Apache NetBeans.
Web Server	Apache Tomcat 8.0 or GlassFish Server 4.1.
Web Browser	Any modern browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) for testing and viewing output.

2.4 Task 1: Data Sharing in Servlet

Objective: To use servlet for request forwarding and data sharing.
Problem Description: Write a login form and a servlet to authenticate a user.
Estimated time: 30 minutes

2.4.1 Step 1: Project Setup

1. Create a directory:

```
F:\[YOUR_COURSE_CODE]-[MATRICNUMBER]
```

Note: Replace ***[YOUR_COURSE_CODE]*** *with your course code (e.g: CSA3023/ CSE3023 / CSM3023) and* ***[MATRICNUMBER]*** *with your actual matric number (e.g., SXXXXX). Create this directory only if you have not created it yet.*

2. Go to the directory and create a subdirectory named:

```
Lab 2
```

3. Open your **Apache NetBeans IDE**.

4. Navigate to the top menu and select:

   - **File → New Project...**

5. Select:

   - **Java Web → Web Application**

   Click **Next**.

6. Enter the project details:

   - Project Name: `ServletDataSharing`
   - Project Location: `F:\[YOUR_COURSE_CODE]-[MATRICNUMBER]\Lab 2`

   Click **Next**.

7. Configure the server:

   - Choose **Apache Tomcat** as the server.

   Click **Finish**.

2.4.2 Step 2: Create HTML Login Page

1. You will see your project structure on the left side of your NetBeans editor. Next, you will create an HTML file for a login page.

2. Right-click your project name (`ServletDataSharing`) → **New** → **HTML File**.

3. Name your HTML file as: `login`
   Click **Finish**.

4. Type the HTML markups into your `login.html`.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Page</title>
</head>
<body>
    <h2>Login</h2>
    <form action="LoginServlet" method="POST">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username"><br><br>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password"><br><br>

        <input type="submit" value="Login">
    </form>
</body>
</html>
```

2.4.3 Step 3: Create a Login Servlet

Next, we are going to create a servlet to process the username and password insert by the user on the login form. We start it by creating a new file for our servlet. Follow the steps below:

1. Right-click on the project.

2. Select **New** → **Servlet**.

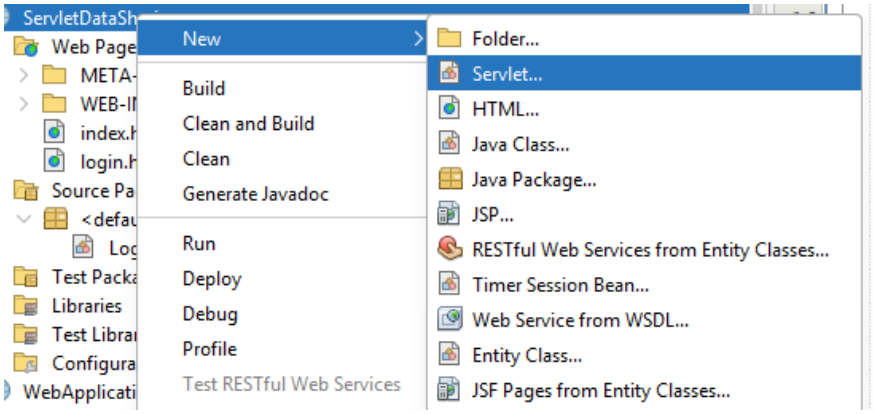

Figure 2.1: Create New Servlet

3. Enter the servlet name: `LoginServlet`.

4. Enter the package name: `SourcePackages`.

5. Click **Finish**.

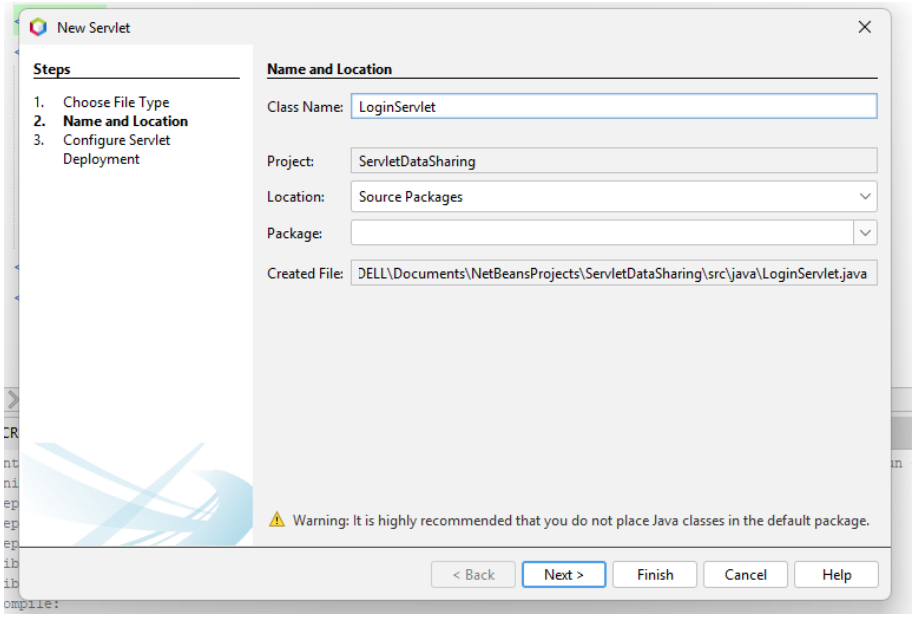

Figure 2.2: Create Login Servlet

6. Configure the servlet deployment by clicking **Add Information in the deployment descriptor.**

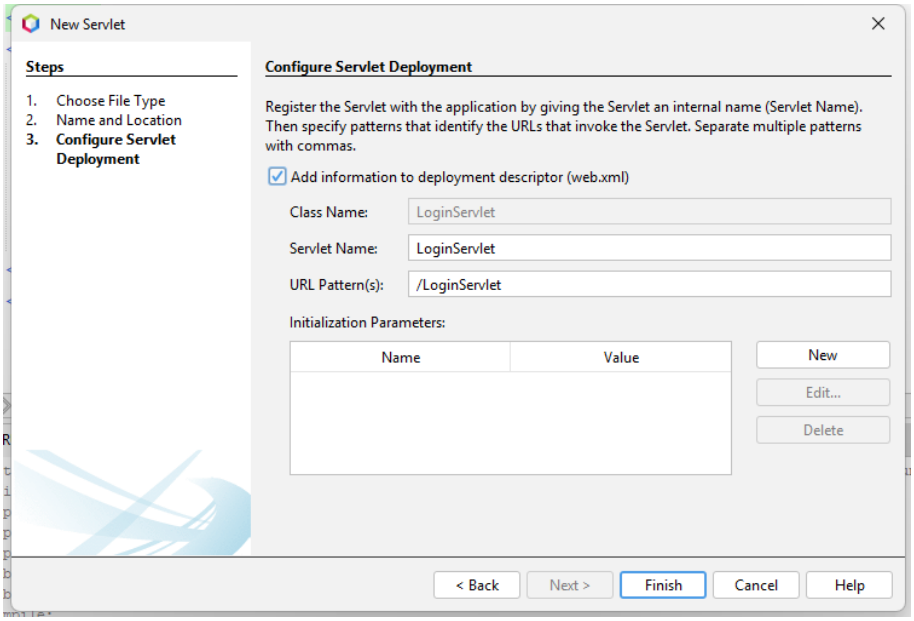

Figure 2.3: Configure servlet deployment

7. Implement the `doPost()` method in `LoginServlet` as follows:

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {

    String user = request.getParameter("username");
    String pass = request.getParameter("password");

    // Semakan kata laluan (Authentication)
    if (user.equals("Ahmad") && pass.equals("4567")) {

        // Data Sharing: Menyimpan maklumat tambahan ke dalam 'request'
        request.setAttribute("accountType", "Premium Student");
        request.setAttribute("email", "ahmad@siswa.edu.my");

        // Request Forwarding ke AccountServlet
        RequestDispatcher rd = request.getRequestDispatcher
        ("AccountServlet");
        rd.forward(request, response);

    } else if (user.equals("Siti") && pass.equals("1234")) {
        // Contoh untuk pengguna lain
        request.setAttribute("accountType", "Standard Student");
        request.setAttribute("email", "siti@siswa.edu.my");
```

```
            RequestDispatcher rd = request.getRequestDispatcher
            ("AccountServlet");
            rd.forward(request, response);

        } else {
            // Jika login gagal
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            out.println("<h3 style='color:red'>Login
            Failed! Invalid credentials.</h3>");
            out.println("<a href='login.html'>Try Again</a>");
        }
    }
```

2.4.4 Step 4: Create an Account Servlet

Next, create a servlet to retrieve users' account information and display it in the browser. Repeat the previous steps to create a new servlet named `AccountServlet`, and configure the details as shown below:

```
//Servlet AccountServle
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;


public class AccountServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Mengambil semula parameter dari borang HTML asal
        String username = request.getParameter("username");
```

```
        // Mengambil data yang dikongsi oleh LoginServlet
        String accountType = (String) request.getAttribute("accountType");
        String email = (String) request.getAttribute("email");

        out.println("<html><body>");
        out.println("<h2>Account Information</h2>");
        out.println("<hr>");
        out.println("<p><strong>Welcome, " + username + "!</strong></p>");
        out.println("<p><b>Email:</b> " + email + "</p>");
        out.println("<p><b>Account Type:</b> " + accountType + "</p>");
        out.println("<br>");
        out.println("<a href='login.html'>Logout</a>");
        out.println("</body></html>");
    }
}
```

2.4.5 Step 5: Run the Project

1. Run the project. The output will be displayed in a web browser. Enter the username `Ahmad` and the password `4567`, then click `Login`.

2. If all previous steps have been completed correctly, the account information for the user `Ahmad` will be displayed. Repeat Step 1 using other user credentials and observe the results.

Reflections

1. What have you learned from this exercise?

2. What are the common methods used in Java Servlet?

2.5 Task 2: Creating A Table in MySQL Database

Objective: To create a MySQL table to store user credentials.
Problem Description: Prepare a user table to be used in Web Application.
Estimated time: 30 minutes

2.5.1 Step 1: Start Apache and MySQL

1. Open the **XAMPP Control Panel**.
2. Click **Start** for both **Apache** and **MySQL** services.

2.5.2 Step 2: Create a New Database

1. Click the **Admin** button for MySQL.
2. Create a new database schema using **your course code**.

   Example: *CSE3023*

2.5.3 Step 3: Create a New MySQL Table

1. After the database has been created, we will now create a table named *users*. This table consists of four columns.
2. Open the **SQL** tab in phpMyAdmin, then execute the following command by clicking the **Go** button:

```
USE CSE3023;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100),
    password VARCHAR(100),
    roles VARCHAR(50)
);
```

3. The newly created table will appear on the left panel in phpMyAdmin.
4. Insert some sample data into the *users* table.
5. After executing the SQL command, phpMyAdmin will display an autogenerated query. To view the inserted data, click the **Browse** tab.

6. The records stored in the *users* table will be displayed. This indicates that the table has been successfully created and populated.
   Note: In a real-world application, passwords must be securely encrypted (e.g., hashed) and should never be stored or displayed in plain text.

2.6 Task 3: Setting the Environment of Web Application for Database Connection

Objective: To set up a proper environment for integrating web application to the database.
Problem Description: Import MySQL JDBC Library to an existing project.
Estimated time: 5 minutes

2.6.1 Step 1: Create a New Web Application Project

1. Open **NetBeans IDE**.
2. Create a new web application project and name it `CRUDusingServlet`.
3. Select **Apache Tomcat** as the application server.
4. Choose **Java EE Web** as the project type.
5. Click **Next**, then click **Finish**. The web application project will be created successfully.

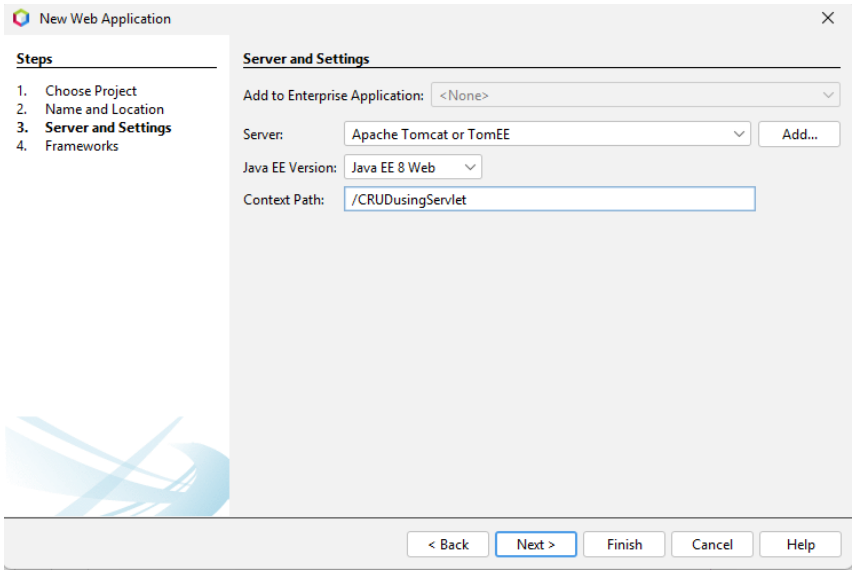

Figure 2.4: Creating the "CRUDusingServlet" Project

2.6.2 Step 2: Prepare the MySQL Connector

1. Open your web browser and navigate to the official MySQL Connector/J download page:

```
https://dev.mysql.com/downloads/connector/j/
```

2. From the **Operating System** drop-down menu, select **Platform Independent**.

3. Click the **Download** button for the ZIP archive file. If prompted to log in or sign up, click the **"No thanks, just start my download."** link at the bottom of the page.

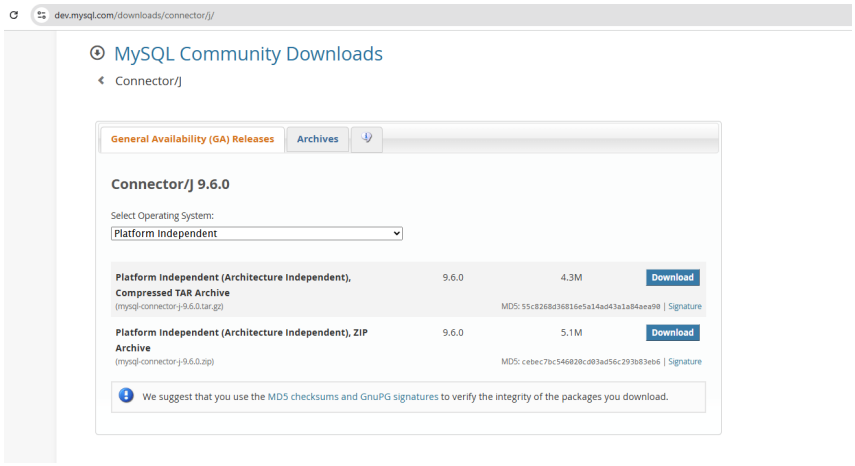

Figure 2.5: Downloading MySQL Connector/J

4. Once the file has been downloaded, extract the ZIP archive to a known location on your computer (e.g., `C:\MySQL-Connector\`).

5. Open the extracted folder and locate the JAR file (e.g., `mysql-connector-j-9.x.x.jar`).

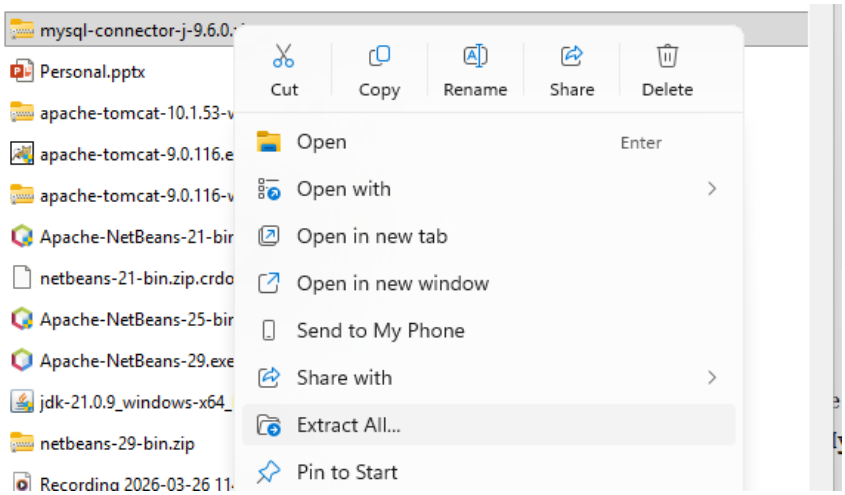

Figure 2.6: Extracting the MySQL Connector/J Archive

6. Return to **NetBeans IDE**.

7. In the **Projects** window, locate and expand your project folder.

8. Right-click on the **Libraries** node and select **Add JAR/Folder...**.

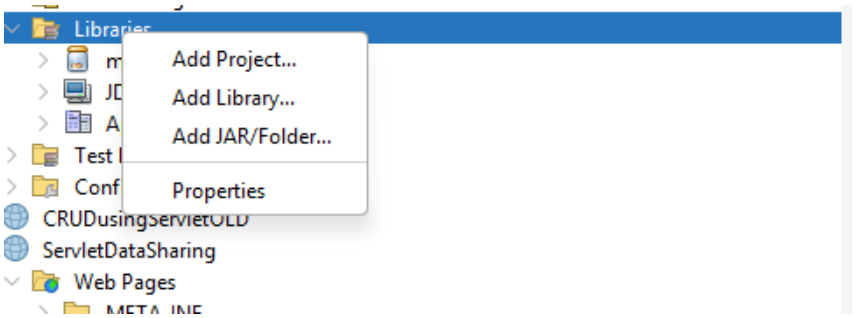

Figure 2.7: Adding External Library to Project

9. In the file chooser window, navigate to the extracted MySQL Connector folder.

10. Select the JAR file (e.g., `mysql-connector-j-9.x.x.jar`) and click **Open**.

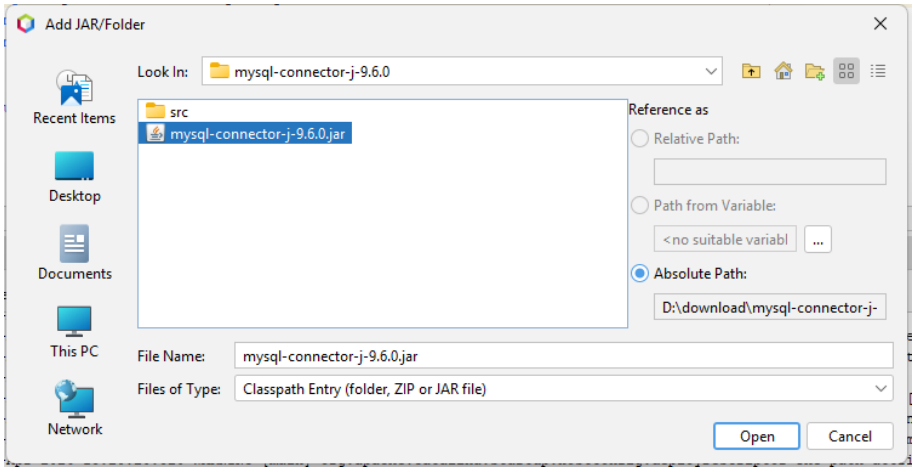

Figure 2.8: Importing MySQL Connector/J Library

11. To verify the installation, expand the **Libraries** node in your project. The MySQL Connector JAR file should now be listed.

12. The application is now ready to connect to the MySQL database and execute SQL queries.

2.7 Task 4: Using Servlets for Database CRUD Operations

Objective: To implement Create, Read, Update, and Delete operations within Servlet methods using the JDBC API.
Problem Description: Develop a web application with Servlets to manage records (insert, view, update, and delete) in a MySQL database.
Estimated time: 60 minutes

2.7.1 Step 1: Create HTML Form for Data Entry

1. In the **Projects** window, right-click on the **Web Pages** folder → **New** → **HTML File...**.

2. Name the file `index` and click **Finish**.

3. Open the `index.html` file and create an HTML form to capture user data, including *Username*, *Password*, and *Roles*.

4. Configure the form by setting the `action` attribute to point to the servlet (e.g., `action="InsertServlet"`) and set the `method` to `POST`.

5. Add a **Submit** button to allow users to send the data to the server for processing.

2.7.2 Step 2: Create the Insert Servlet (Create)

1. In the **Source Packages** node, right-click → **New** → **Servlet...**.
2. Name the servlet `InsertServlet`, assign it to an appropriate package (e.g., `com.lab.controller`), and click **Finish**.
3. In the `doPost()` method, retrieve the form data using `request.getParameter()` for the fields `username`, `password`, and `roles`.
4. Load the MySQL JDBC driver using `Class.forName(...)` and establish a database connection using `DriverManager.getConnection()` with your database (e.g., `CSE3023`).
5. Create a `PreparedStatement` to execute the following SQL query:

```
INSERT INTO users (username, password, roles) VALUES (?, ?, ?)
```

6. Set the retrieved form values into the query parameters and execute the statement using `executeUpdate()`.
7. Upon successful insertion, redirect the user to the data listing page using:

```
response.sendRedirect("ViewServlet");
```

8. The complete code snippet for the `doPost()` method in `InsertServlet` is shown below:

```
package com.lab.controller;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class InsertServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {
```

```
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        String roles = request.getParameter("roles");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection("jdbc:mysql://
            localhost:3306/CSE3023", "root", "admin");

            String sql = "INSERT INTO users (username, password, roles)
            VALUES (?, ?, ?)";
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, username);
            pstmt.setString(2, password);
            pstmt.setString(3, roles);
            pstmt.executeUpdate();

            pstmt.close();
            conn.close();

            response.sendRedirect("ViewServlet");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2.7.3 Step 3: Create the View Servlet (Read)

1. Create a new servlet and name it `ViewServlet`.

2. In the `doGet()` method, establish a connection to the database in the same way as in Step 2.

3. Prepare a SQL query to fetch all user records from the database table:

```
SELECT * FROM users;
```

4. Execute the query using `executeQuery()` and store the results in a `ResultSet` object.

5. Iterate through the `ResultSet` using a `while (rs.next())` loop to access each record.

6. Dynamically generate an HTML table using `out.println()` to display the retrieved data (ID, Username, Password, Roles) row by row.

7. **Important:** Add two additional columns in the HTML table for *Edit* and *Delete* actions. These should be hyperlinks pointing to `UpdateServlet` and `DeleteServlet`, passing the record's ID in the URL. **Example:**

```
<a href='DeleteServlet?id=1'>Delete</a>
<a href='UpdateServlet?id=1'>Edit</a>
```

8. The complete code snippet for the `doGet()` method in `ViewServlet` is shown below:

```
package com.lab.controller;


import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;


public class ViewServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```
        out.println("<h2>User List</h2>");
        out.println("<table border='1'><tr><th>ID</th><th>Username</th><th>
        Password </th><th>Role</th><th>Actions</th></tr>");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection("jdbc:mysql://
            localhost:3306/CSE3023", "root", "admin");

            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM users");

            while (rs.next()) {
                int id = rs.getInt("id");
                out.println("<tr>");
                out.println("<td>" + id + "</td>");
                out.println("<td>" + rs.getString("username") + "</td>");
                // Untuk lab, kita paparkan password. Untuk sistem sebenar,
                jangan paparkan!
                out.println("<td>" + rs.getString("password") + "</td>");
                out.println("<td>" + rs.getString("roles") + "</td>");
                out.println("<td><a href='UpdateServlet?id=" + id + "'>
                Edit</a> | ");
                out.println("<a href='DeleteServlet?id=" + id + "'>
                Delete</a></td>");
                out.println("</tr>");
            }
            out.println("</table>");
            out.println("<br><a href='index.html'>Add New User</a>");

            rs.close();
            stmt.close();
            conn.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2.7.4 Step 4: Create the Delete Servlet (Delete)

1. Create a new servlet and name it `DeleteServlet`.

2. In the `doGet()` method, retrieve the record ID passed from the URL using:

```
String id = request.getParameter("id");
```

3. Establish a database connection and prepare a SQL statement to delete the record:

```
DELETE FROM users WHERE id = ?
```

4. Set the retrieved ID as the parameter for the `PreparedStatement` and execute the query using `executeUpdate()`.

5. After successful deletion, redirect the user back to the updated table view:

```
response.sendRedirect("ViewServlet");
```

6. The complete code snippet for the `doGet()` method in `DeleteServlet` is shown below:

```
public class DeleteServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

        String id = request.getParameter("id");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection("jdbc:mysql://
            localhost:3306/CSE3023", "root", "admin");

            String sql = "DELETE FROM users WHERE id=?";
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setInt(1, Integer.parseInt(id));
            pstmt.executeUpdate();

            pstmt.close();
            conn.close();

            response.sendRedirect("ViewServlet");
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2.7.5 Step 5: Create the Update Servlet (Update)

1. Create a new servlet and name it `UpdateServlet`.
2. In the `doGet()` method:
    - Retrieve the record ID from the URL using `request.getParameter("id")`.
    - Fetch the corresponding user record from the database.
    - Populate an HTML form with the retrieved data so the user can edit the existing values.
    - Set the form's `action` attribute to point to `UpdateServlet` and use the `POST` method for submission.

```
//UpdateServlet doGet() Method: Display Form for Editing Record
      // Paparkan borang dengan data sedia ada
protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String id = request.getParameter("id");

    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection("jdbc:mysql://
        localhost:3306/CSE3023", "root", "admin2128");

        String sql = "SELECT * FROM users WHERE id=?";
        PreparedStatement pstmt = conn.prepareStatement(sql);
        pstmt.setInt(1, Integer.parseInt(id));
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
```

```
            String currentRole = rs.getString("roles");

            out.println("<h2>Update User</h2>");
            out.println("<form action='UpdateServlet' method='POST'>");
            out.println("<input type='hidden' name='id' value='" +
            rs.getInt("id") + "'>");
            out.println("Username: <input type='text' name='username'
            value='" + rs.getString("username") + "' required><br><br>");
            out.println("Password: <input type='text' name='password'
            value='" + rs.getString("password") + "' required><br>
            <br>");
            out.println("Role: <select name='roles'>");
            out.println("<option value='Admin' " + (currentRole.equals
            ("Admin") ? "selected" : "") + ">Admin</option>");
            out.println("<option value='Staff' " + (currentRole.equals
            ("Staff") ? "selected" : "") + ">Staff</option>");
            out.println("<option value='Student' " +
            (currentRole.equals("Student") ? "selected" : "") + ">
            Student</option>");
            out.println("</select><br><br>");
            out.println("<input type='submit' value='Update User'>");
            out.println("</form>");
        }

        rs.close();
        pstmt.close();
        conn.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

3. In the `doPost()` method:

   - Retrieve the modified form data (`username`, `password`, `roles`).
   - Execute an SQL `UPDATE` statement to save the changes:

     ```
     UPDATE users SET username=?, password=?, roles=? WHERE id=?
     ```

   - Redirect the user back to `ViewServlet` to display the updated table.

```
//UpdateServlet.doPost() Method: Save Edited Record to Database
     // Proses dan simpan data yang telah dikemaskini
   protected void doPost(HttpServletRequest request, HttpServletResponse
   response) throws ServletException, IOException {

       String id = request.getParameter("id");
       String username = request.getParameter("username");
       String password = request.getParameter("password");
       String roles = request.getParameter("roles");

       try {
           Class.forName("com.mysql.cj.jdbc.Driver");
           Connection conn = DriverManager.getConnection("jdbc:mysql://
           localhost:3306/CSE3023", "root", "admin");

           String sql = "UPDATE users SET username=?, password=?, roles=?
           WHERE id=?";
           PreparedStatement pstmt = conn.prepareStatement(sql);
           pstmt.setString(1, username);
           pstmt.setString(2, password);
           pstmt.setString(3, roles);
           pstmt.setInt(4, Integer.parseInt(id));
           pstmt.executeUpdate();

           pstmt.close();
           conn.close();

           response.sendRedirect("ViewServlet");

       } catch (Exception e) {
           e.printStackTrace();
       }
   }
```

2.8 Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations

Objective: To refactor the existing CRUD web application by implementing the Data

Access Object (DAO) pattern and JavaBeans for better code organization and separation of concerns.

Problem Description: Modify the previous User Management application (from Task 4) by removing all direct database logic from the Servlets. Instead, you will create a dedicated DAO class to handle database queries and use a JavaBean to represent the User entity.

Estimated time: 60 minutes

2.8.1 Step 1: Copy and Rename the Project

1. In the NetBeans **Projects** window, right-click on your previous project from Task 4.
2. Select **Copy...** from the context menu.
3. In the **Copy Project** dialog, change the Project Name to `CRUDusingServletJavabeanDAO` and select your desired project location.
4. Click **Copy** to duplicate the project. You will now work within this new project for the remainder of this task.

2.8.2 Step 2: Create the User JavaBean (Model)

1. In the **Projects** window, right-click on the **Source Packages** folder inside the new project `CRUDusingServletJavabeanDAO` → **New** → **Java Class...**.
2. Name the class `User` and create a new package `com.lab.model`. Click **Finish**.
3. In the `User.java` file, declare private attributes that correspond to your database columns:
   - `id` (int)
   - `username` (String)
   - `password` (String)
   - `roles` (String)
4. Create a default (empty) constructor and a parameterized constructor for the class.
5. Generate **Getter and Setter** methods for all attributes.
   *Tip: In NetBeans, right-click inside the class editor → **Insert Code... → Getter and Setter...** to generate them automatically.*

```
//User JavaBean Clas
    package com.lab.model;

public class User {
    private int id;
    private String username;
    private String password;
    private String roles;

    // Constructor Kosong (Wajib untuk JavaBean)
    public User() {
    }

    // Constructor dengan ID (Untuk Update & Delete)
    public User(int id, String username, String password, String roles) {
        this.id = id;
        this.username = username;
        this.password = password;
        this.roles = roles;
    }

    // Constructor tanpa ID (Untuk Insert)
    public User(String username, String password, String roles) {
        this.username = username;
        this.password = password;
        this.roles = roles;
    }

    // Getter dan Setter
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }
```

```
    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRoles() {
        return roles;
    }

    public void setRoles(String roles) {
        this.roles = roles;
    }
}
```

2.8.3 Step 3: Create the Data Access Object (UserDAO)

1. Create another Java Class named `UserDAO` inside a new package called `com.lab.dao`.
2. Create a helper method (e.g., `protected Connection getConnection()`) inside this class to establish the JDBC connection to the `CSE3023` database.
3. Migrate the SQL CRUD operations from your previous Servlets into this DAO class. You need to create methods such as:
   - `public void insertUser(User user)`
   - `public List<User> selectAllUsers()`
   - `public boolean deleteUser(int id)`
   - `public boolean updateUser(User user)`
   - `public User selectUser(int id)`
4. Use the `User` JavaBean to pass data into these methods and to return data from the database.

```java
//class userDAO
package com.lab.dao;

import com.lab.model.User;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class UserDAO {
    private String jdbcURL = "jdbc:mysql://localhost:3306/CSE3023";
    private String jdbcUsername = "root";
    private String jdbcPassword = "admin";

    // Method untuk mendapatkan sambungan Database
    protected Connection getConnection() {
        Connection connection = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            connection = DriverManager.getConnection(jdbcURL,
            jdbcUsername, jdbcPassword);
        } catch (SQLException | ClassNotFoundException e) {
            e.printStackTrace();
        }
        return connection;
    }

    // CREATE: Masukkan pengguna baharu
    public void insertUser(User user) {
        String sql = "INSERT INTO users (username, password, roles)
        VALUES (?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, user.getUsername());
            pstmt.setString(2, user.getPassword());
            pstmt.setString(3, user.getRoles());
```

```
            pstmt.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // READ: Ambil satu pengguna berdasarkan ID (Untuk Update)
    public User selectUser(int id) {
        User user = null;
        String sql = "SELECT id, username, password, roles FROM users
        WHERE id =?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, id);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                String username = rs.getString("username");
                String password = rs.getString("password");
                String roles = rs.getString("roles");
                user = new User(id, username, password, roles);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return user;
    }

    // READ: Ambil senarai semua pengguna (Untuk View)
    public List<User> selectAllUsers() {
        List<User> users = new ArrayList<>();
        String sql = "SELECT * FROM users";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                int id = rs.getInt("id");
```

```
            String username = rs.getString("username");
            String password = rs.getString("password");
            String roles = rs.getString("roles");
            users.add(new User(id, username, password, roles));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return users;
}

// UPDATE: Kemaskini maklumat pengguna
public boolean updateUser(User user) {
    boolean rowUpdated = false;
    String sql = "UPDATE users SET username = ?, password = ?,
    roles = ? WHERE id = ?";
    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, user.getUsername());
        pstmt.setString(2, user.getPassword());
        pstmt.setString(3, user.getRoles());
        pstmt.setInt(4, user.getId());

        rowUpdated = pstmt.executeUpdate() > 0;

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return rowUpdated;
}

// DELETE: Padam pengguna
public boolean deleteUser(int id) {
    boolean rowDeleted = false;
    String sql = "DELETE FROM users WHERE id = ?";
    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, id);
        rowDeleted = pstmt.executeUpdate() > 0;
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return rowDeleted;
    }
}
```

2.8.4 Step 4: Modify the Existing Servlets (Controller)

1. Open your existing Servlets (e.g., `InsertServlet`, `ViewServlet`, `UpdateServlet`, `DeleteServlet`).
2. Remove all the JDBC-related code (`DriverManager`, `Connection`, `PreparedStatement`, `ResultSet`) from these Servlets.
3. Instantiate the `UserDAO` class inside your Servlets.
4. Modify the Servlets to call the DAO methods. For example, in `InsertServlet`, capture the form parameters, create a new `User` object, and pass it to `userDAO.insertUser(newUser)`.
5. In `ViewServlet`, call `userDAO.selectAllUsers()` to retrieve a list of users, and iterate through that list to generate the HTML table.
6. Run and test your application to ensure all Create, Read, Update, and Delete functions work exactly as before, but now running through the DAO architecture.

```
 // Class InsertServlet
package com.lab.controller;

import com.lab.dao.UserDAO;
import com.lab.model.User;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;


public class InsertServlet extends HttpServlet {
```

```
    private UserDAO userDAO;

    public void init() {
        userDAO = new UserDAO();
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");
        String roles = request.getParameter("roles");

        User newUser = new User(username, password, roles);
        userDAO.insertUser(newUser); // Panggil DAO, bukan JDBC

        response.sendRedirect("ViewServlet");
    }
}
```

```
// Class ViewServlet
package com.lab.controller;

import com.lab.dao.UserDAO;
import com.lab.model.User;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;


public class ViewServlet extends HttpServlet {
    private UserDAO userDAO;

    public void init() {
        userDAO = new UserDAO();
    }
```

```
    protected void doGet(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        List<User> listUser = userDAO.selectAllUsers(); // Panggil DAO

        out.println("<h2>User List (Using DAO)</h2>");
        out.println("<table border='1'><tr><th>ID</th><th>Username</th>
        <th>Password</th><th>Role</th><th>Actions</th></tr>");

        for (User user : listUser) {
            out.println("<tr>");
            out.println("<td>" + user.getId() + "</td>");
            out.println("<td>" + user.getUsername() + "</td>");
            out.println("<td>" + user.getPassword() + "</td>");
            out.println("<td>" + user.getRoles() + "</td>");
            out.println("<td><a href='UpdateServlet?id=" + user.getId()
            + "'>Edit</a> | ");
            out.println("<a href='DeleteServlet?id=" + user.getId() + "'>
            Delete</a></td>");
            out.println("</tr>");
        }
        out.println("</table>");
        out.println("<br><a href='index.html'>Add New User</a>");
    }
}
```

```
// Class DeleteServlet
package com.lab.controller;

import com.lab.dao.UserDAO;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

```
public class DeleteServlet extends HttpServlet {
    private UserDAO userDAO;

    public void init() {
        userDAO = new UserDAO();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

        int id = Integer.parseInt(request.getParameter("id"));
        userDAO.deleteUser(id); // Panggil DAO

        response.sendRedirect("ViewServlet");
    }
}
```

```
//Class UpdateServlet
package com.lab.controller;

import com.lab.dao.UserDAO;
import com.lab.model.User;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class UpdateServlet extends HttpServlet {
    private UserDAO userDAO;

    public void init() {
        userDAO = new UserDAO();
    }

    // Paparkan borang dengan data sedia ada
    protected void doGet(HttpServletRequest request, HttpServletResponse
```

```
response) throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    int id = Integer.parseInt(request.getParameter("id"));

    User existingUser = userDAO.selectUser(id); // Panggil DAO

    if (existingUser != null) {
        out.println("<h2>Update User (Using DAO)</h2>");
        out.println("<form action='UpdateServlet' method='POST'>");
        out.println("<input type='hidden' name='id' value='" +
        existingUser.getId() + "'>");
        out.println("Username: <input type='text' name='username'
        value='" + existingUser.getUsername() + "' required><br><br>");
        out.println("Password: <input type='text' name='password'
        value='" + existingUser.getPassword() + "' required><br><br>");

        String currentRole = existingUser.getRoles();
        out.println("Role: <select name='roles'>");
        out.println("<option value='Admin' " +
        (currentRole.equals("Admin") ? "selected" : "") + ">
        Admin</option>");
        out.println("<option value='Staff' " +
        (currentRole.equals("Staff") ? "selected" : "") + ">
        Staff</option>");
        out.println("<option value='Student' " +
        (currentRole.equals("Student") ? "selected" : "") + ">
        Student</option>");
        out.println("</select><br><br>");

        out.println("<input type='submit' value='Update User'>");
        out.println("</form>");
    }
}

// Simpan data yang telah dikemaskini
protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {

    int id = Integer.parseInt(request.getParameter("id"));
```

```
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        String roles = request.getParameter("roles");

        User user = new User(id, username, password, roles);
        userDAO.updateUser(user); // Panggil DAO

        response.sendRedirect("ViewServlet");
    }
}
```

2.9 Exercise

Objective: Develop a **Product Inventory System** by applying knowledge of Servlets, JavaBeans, and the DAO pattern to create a simple dynamic web application.

Instruction

Please complete this exercise by fulfilling the following requirements:

1. **Database Setup:** Create a new table named `products` in your `CSE3023` database with the following columns:
    - `id` (INT, Primary Key, Auto Increment)
    - `name` (VARCHAR)
    - `category` (VARCHAR)
    - `price` (DOUBLE)
    - `quantity` (INT)
2. **JavaBean (Model):** Create a class named `Product` in the `com.lab.model` package. Ensure it has the appropriate private attributes, constructors (empty and parameterized), and getter/setter methods.
3. **Data Access Object (DAO):** Create a `ProductDAO` class in the `com.lab.dao` package to handle all SQL operations (`INSERT`, `SELECT`, `UPDATE`, `DELETE`).
4. **Servlets (Controller):** Create the necessary Servlets (e.g., `InsertProductServlet`, `ViewProductServlet`, `UpdateProductServlet`, `DeleteProductServlet`) to capture user inputs and interact with your `ProductDAO`. **Strict rule:** No JDBC code should be written inside these Servlets!

5. **User Interface (View):** Create an HTML form (`add_product.html`) to insert new products. Ensure your view servlet generates an HTML table displaying all products, along with functional "Edit" and "Delete" hyperlinks for each row.

Assessment Rubric (10 Marks)

Criteria	**Marks**
Database & Model: Correct creation of the `products` table in the database and the `Product` JavaBean with appropriate attributes, constructors, and getters/setters.	2
DAO Implementation: Successful implementation of all CRUD SQL operations (`INSERT`, `SELECT`, `UPDATE`, `DELETE`) inside the `ProductDAO` class.	3
Controller/Servlets: Correct creation of Servlets that successfully capture parameters and call DAO methods **without** any direct JDBC code inside them.	2
View/User Interface: Proper HTML form for input and dynamic generation of the product list table, including functional "Edit" and "Delete" links.	2
Execution: Overall correctness, clean code structure, and successful execution of the application without errors.	1
Total	**10**